

ELE 216 Laboratoire 3 : Modules et types opaques

Pour chacun des problèmes suivants, écrivez un module complet : un fichier d'en-tête (.h) qui définit l'interface publique, un fichier source (.c) qui implémente le module avec un type opaque, un fichier de tests unitaires utilisant Unity, et un Makefile. Documentez vos fichiers d'en-tête à mesure en format Doxygen.

Vous devez réaliser ce laboratoire seul, pendant la période. Il vous est permis de discuter avec vos collègues, consulter les chargés de laboratoires et l'internet.

Il est permis de réutiliser certaines fonctions réalisées dans les numéros précédents pour les numéros suivants.

Vous devez faire valider votre travail avant de quitter. On vous demandera d'expliquer clairement le fonctionnement de certains éléments de votre implémentation.

Problème 1 : Pile rechargeable

Modélisez une pile rechargeable avec une capacité bornée. La capacité maximale et la tension nominale sont fixées à la fabrication et ne peuvent pas changer.

État interne :

- capacité maximale en mAh (immuable)
- charge courante en mAh
- tension nominale en V (immuable)

Interface :

`pile_t *pile_creer(float capacite_mAh, float tension_V);` alloue, charge initiale à 0. Retourne NULL si paramètres invalides.

`void pile_detruire(pile_t *p);` libère la mémoire.

`void pile_charger(pile_t *p, float mAh);` ajoute mAh à la charge. La charge est saturée à la capacité maximale.

`void pile_decharger(pile_t *p, float mAh);` retire mAh de la charge. La charge ne descend jamais sous 0.

`float pile_capacite_max(const pile_t *p);` retourne la capacité maximale.

`float pile_charge_courante(const pile_t *p);` retourne la charge courante en mAh.

`float pile_tension(const pile_t *p) ;` retourne la tension nominale.

`float pile_pourcentage(const pile_t *p) ;` retourne charge / capacité_max × 100.

`void pile_afficher(const pile_t *p) ;` affiche tous les champs.

Tests unitaires à inclure :

- création valide / invalide (capacité ou tension ≤ 0)
- charge partielle puis lecture, valeur correcte
- saturation à la capacité maximale
- décharge complète sans descendre sous 0
- calcul du pourcentage à plusieurs niveaux (0 %, 50 %, 100 %)

Problème 2 : Buffer circulaire (FIFO)

Modélisez une file FIFO d'entiers de capacité fixe, implémentée sur un tableau circulaire. La capacité est définie à la création et ne peut pas changer.

État interne :

- capacité (immuable)
- pointeur vers un tableau d'entiers sur le heap (malloc)
- index de la tête (prochain élément à défiler)
- index de la queue (prochain emplacement libre)
- nombre d'éléments présents

Interface :

`buffer_t *buffer_creer(int capacite) ;` alloue le buffer ET son tableau interne. Retourne NULL si capacité ≤ 0 .

`void buffer_detruire(buffer_t *b) ;` libère le buffer ET son tableau interne.

`int buffer_enfiler(buffer_t *b, int valeur) ;` ajoute en queue. Retourne 1 si succès, 0 si plein.

`int buffer_defiler(buffer_t *b, int *valeur) ;` retire le plus ancien dans *valeur. Retourne 1 si succès, 0 si vide.

`int buffer_capacite(const buffer_t *b) ;` retourne la capacité.

`int buffer_taille(const buffer_t *b) ;` retourne le nombre d'éléments présents.

`int buffer_est_vide(const buffer_t *b) ;` retourne 1 si vide, 0 sinon.

`int buffer_est_plein(const buffer_t *b) ;` retourne 1 si plein, 0 sinon.

`void buffer_afficher(const buffer_t *b) ;` affiche le contenu en ordre FIFO.

Tests unitaires à inclure :

- création valide / invalide
- enfilage et défilage simples : l'ordre FIFO est respecté
- défiler depuis un buffer vide retourne 0
- enfiler dans un buffer plein retourne 0 et ne modifie rien
- comportement correct sur plusieurs cycles complets remplir/vider (wrap-around) (faites un petit buffer de 3) (si la tâche n'est pas claire, demandez au chargé de labo).

Problème 3 : Filtre moyenne mobile

Modélisez un filtre passe-bas par moyenne mobile sur une fenêtre d'échantillons. La taille de la fenêtre est fixée à la création. Avant que la fenêtre soit remplie, la moyenne porte uniquement sur les échantillons disponibles.

État interne :

- taille de la fenêtre (immuable)
- pointeur vers un tableau de float sur le heap (malloc)
- nombre d'échantillons reçus depuis la création
- index du prochain emplacement à écrire

Interface :

`filtre_t *filtre_creer(int taille_fenetre) ;` alloue le filtre ET son tableau interne. Retourne NULL si taille ≤ 0 .

`void filtre_detruire(filtre_t *f) ;` libère le filtre ET son tableau interne.

`void filtre_ajouter(filtre_t *f, float echantillon) ;` ajoute un échantillon. Lorsque la fenêtre est pleine, le plus ancien est remplacé.

`float filtre_valeur(const filtre_t *f) ;` retourne la moyenne des échantillons disponibles. Retourne 0 si aucun échantillon n'a été ajouté.

`int filtre_taille_fenetre(const filtre_t *f);` retourne la taille de la fenêtre.

`int filtre_nb_echantillons(const filtre_t *f);` retourne le nombre d'échantillons dans la fenêtre courante (saturé à la taille).

`void filtre_afficher(const filtre_t *f);` affiche les échantillons dans la fenêtre et la moyenne courante.

Tests unitaires à inclure :

- création valide / invalide
- valeur retournée avant tout échantillon = 0
- moyenne partielle (moins d'échantillons que la fenêtre)
- moyenne sur fenêtre exactement remplie
- débordement : le plus ancien échantillon est bien remplacé
- signal constant donne une moyenne constante (à la précision flottante près)